# AI Security Findings

Overall risk: LOW | Total findings: 1

## Executive summary

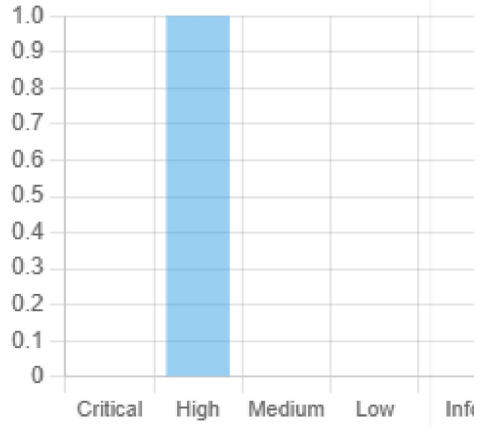
### Risk overview

Total findings: 1. Critical: 0, High: 1, Medium: 0, Low: 0.

### Technical summary

[
{
"title": "Potential Reflected/Stored Cross-Site Scripting (XSS) Vulnerabilities via Uploaded Filename",
"severity": "high",
"location": "xss_report_probe.py",
"description": "Uploaded filenames containing crafted XSS payloads are reflected unsafely in HTML responses at the /scan/file endpoint. This includes dangerous tags, inline event handlers, javascript: URIs, and template injections, indicating insufficient input sanitization or encoding. Such vulnerabilities expose users to arbitrary JavaScript execution via stored or reflected XSS attacks.",
"recommendation": "Implement rigorous input validation by enforcing a whitelist of acceptable filename characters. Apply contextual output encoding (HTML entity encoding) on any reflected user input. Reject or sanitize dangerous characters in filenames before rendering. Conduct thorough manual and automated testing to ensure all XSS vectors are mitigated.",
"source_file": "/tmp/tmpmf6gzdiv/xss_report_probe.py",
"owasp_category": "A07:2021 - Cross-Site Scripting (XSS)"
}
]

### Severity distribution



| Severity | Title | File / Location | Description | Recommendation |
|---|---|---|---|---|
| **HIGH** | Potential Reflected/Stored Cross-Site Scripting (XSS) Vulnerabilities via Uploaded Filename | `xss_report_probe.py` | The code uploads files with crafted filenames containing various XSS payloads to the /scan/file endpoint on the target server. It then analyzes the response HTML for unsafe reflections of these filenames. The code evidences reflections of dangerous tags (e.g., <script>), inline event handlers (e.g., onerror), javascript: URIs, and template injections, indicating that the application under test may improperly sanitize or encode user-controllable input. This facilitates stored or reflected XSS attacks, which can lead to execution of arbitrary JavaScript in users' browsers. | The target application must properly sanitize, encode, or reject dangerous characters in uploaded filenames before reflecting them in HTML responses. Use contextual output encoding (e.g., HTML entity encoding) on reflected data and implement a whitelist of acceptable characters for filenames. Perform extensive manual and automated testing to confirm sanitization robustness and fix any XSS vulnerabilities found. |